

上述算法过程，可具体实现如代码8.5所示。

```

1 template <typename T> bool Splay<T>::remove(const T& e) { //从伸展树中删除关键码e
2     if (!_root || (e != search(e)->data)) return false; //若树空或目标关键码不存在，则无法删除
3     BinNodePosi(T) w = _root; //assert: 经search()后节点e已被伸展至树根
4     if (!HasLChild(*_root)) { //若无左子树，则直接删除
5         _root = _root->rChild; if (_root) _root->parent = NULL;
6     } else if (!HasRChild(*_root)) { //若无右子树，也直接删除
7         _root = _root->lChild; if (_root) _root->parent = NULL;
8     } else { //若左右子树同时存在，则
9         BinNodePosi(T) lTree = _root->lChild;
10        lTree->parent = NULL; _root->lChild = NULL; //暂时将左子树切除
11        _root = _root->rChild; _root->parent = NULL; //只保留右子树
12        search(w->data); //以原树根为目标，做一次（必定失败的）查找
13        //assert: 至此，右子树中最小节点必伸展至根，且（因无雷同节点）其左子树必空，于是
14        _root->lChild = lTree; lTree->parent = _root; //只需将原左子树接回原位即可
15    }
16    release(w->data); release(w); _size--; //释放节点，更新规模
17    if (_root) updateHeight(_root); //此后，若树非空，则树根的高度需要更新
18    return true; //返回成功标志
19 } //若目标节点存在且被删除，返回true；否则返回false

```

代码8.5 伸展树节点的删除

当然，其中的第二次查找也可在 T_L （若非空）中进行。读者不妨独立实现这一对称的版本。

§ 8.2 B-树

8.2.1 多路平衡查找

■ 分级存储

现代电子计算机发展速度空前。就计算能力而言，ENIAC^②每秒只能够执行5000次加法运算，而今天的超级计算机每秒已经能够执行 3×10^{16} 次以上的浮点运算^③。就存储能力而言，情况似乎也是如此：ENIAC只有一万八千个电子管，而如今容量以TB计的硬盘也不过数百元，内存的常规容量也已达GB量级。

然而从实际应用的需求来看，问题规模的膨胀却远远快于存储能力的增长。以数据库为例，在20世纪80年代初，典型数据库的规模为10~100 MB，而三十年后的今天，典型数据库的规模已需要以TB为单位来计量。计算机存储能力提高速度相对滞后，是长期存在的现象，而且随着时间的推移，这一矛盾将日益凸显。鉴于在同等成本下，存储器的容量越大（小）则访问速度越慢（快），因此一味地提高存储器容量，亦非解决这一矛盾的良策。

^② 第一台电子计算机，1946年2月15日诞生于美国宾夕法尼亚大学工学院

^③ 2013年6月，天河-2以此运算速度，荣登世界超级计算机500强榜首

实践证明,分级存储才是行之有效的方法。在由内存与外存(磁盘)组成的二级存储系统中,数据全集往往存放于外存中,计算过程中则可将内存作为外存的高速缓存,存放最常用数据项的复本。借助高效的调度算法,如此便可将内存的“高速度”与外存的“大容量”结合起来。

两个相邻存储级别之间的数据传输,统称I/O操作。各级存储器的访问速度相差悬殊,故应尽可能地减少I/O操作。仍以内存与磁盘为例,其单次访问延迟大致分别在纳秒(ns)和毫秒(ms)级别,相差5至6个数量级。也就是说,对内存而言的一秒/一天,相当于磁盘的一星期/两千年。因此,为减少对外存的一次访问,我们宁愿访问内存百次、千次甚至万次。也正因为此,在衡量相关算法的性能时,基本可以忽略对内存的访问,转而更多地关注对外存的访问次数。

■ 多路搜索树

当数据规模大到内存已不足以容纳时,常规平衡二叉搜索树的效率将大打折扣。其原因在于,查找过程对外存的访问次数过多。例如,若将 10^9 个记录在外存中组织为AVL树,则每次查找大致需做30次外存访问。那么,如何才能有效减少外存操作呢?

为此,需要充分利用磁盘之类外部存储器的另一特性:就时间成本而言,读取物理地址连续的一千个字节,与读取单个字节几乎没有区别。既然外部存储器更适宜于批量式访问,不妨通过时间成本相对极低的多次内存操作,来替代时间成本相对极高的单次外存操作。相应地,需要将通常的二叉搜索树,改造为多路搜索树——在中序遍历的意义下,这也是一种等价变换。

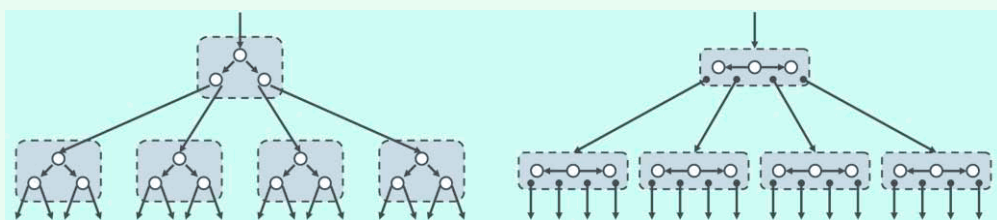


图8.10 二叉搜索树与四路搜索树

具体地如图8.10所示,比如可以两层为间隔,将各节点与其左、右孩子合并为“大节点”:原节点及其孩子的共三个关键码予以保留;孩子节点原有的四个分支也予以保留并按中序遍历次序排列;节点到左、右孩子的分支转化为“大节点”内部的搜索,在图中表示为水平分支。如此改造之后,每个“大节点”拥有四个分支,故称作四路搜索树。

这一策略还可进一步推广,比如以三层为间隔,将各节点及其两个孩子、四个孙子合并为含有七个关键码、八个分支的“大节点”,进而得到八路搜索树。一般地,以 k 层为间隔如此重组,可将二叉搜索树转化为等价的 2^k 路搜索树,统称多路搜索树(multi-way search tree)。

不难验证,多路搜索树同样支持查找等操作,且效果与原二叉搜索树完全等同;然而重要的是,其对外存的访问方式已发生本质变化。实际上,在此时的搜索每下降一层,都以“大节点”为单位从外存读取一组(而不再是单个)关键码。更为重要的是,这组关键码在逻辑上与物理上都彼此相邻,故可以批量方式从外存一次性读出,且所需时间与读取单个关键码几乎一样。

当然,每组关键码的最佳数目,取决于不同外存的批量访问特性。比如旋转式磁盘的读写操作多以扇区为单位,故可根据扇区的容量和关键码的大小,经换算得出每组关键码的最佳规模。例如若取 $k = 8$,则每个“大节点”将拥有255个关键码和256个分支,此时同样对于1G个记录,每次查找所涉及的外存访问将减至4~5次。

■ 多路平衡搜索树

所谓 m 阶B-树^④ (B-tree)，即 m 路平衡搜索树 ($m \geq 2$)，其宏观结构如图8.11所示。

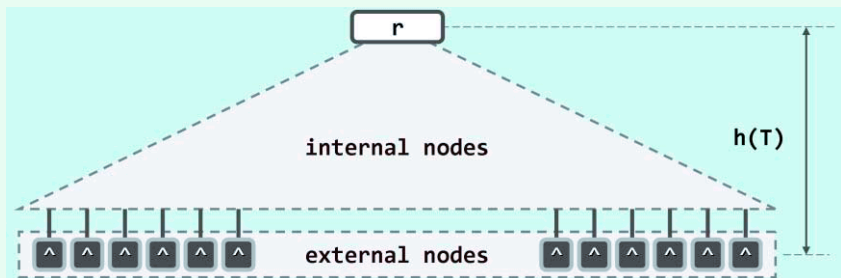


图8.11 B-树的宏观结构（外部节点以深色示意，深度完全一致，且都同处于最底层）

其中，所有外部节点均深度相等。同时，每个内部节点都存有不超过 $m - 1$ 个关键码，以及用以指示对应分支的不超过 m 个引用。具体地，存有 $n \leq m - 1$ 个关键码：

$$K_1 < K_2 < K_3 < K_4 < \dots < K_n$$

的内部节点，同时还配有 $n + 1 \leq m$ 个引用：

$$A_0 < A_1 < A_2 < A_3 < A_4 < \dots < A_n$$

反过来，各内部节点的分支数也不能太少。具体地，除根以外的所有内部节点，都应满足：

$$n + 1 \geq \lceil m/2 \rceil$$

而在非空的B-树中，根节点应满足：

$$n + 1 \geq 2$$

由于各节点的分支数介于 $\lceil m/2 \rceil$ 至 m 之间，故 m 阶B-树也称作 $(\lceil m/2 \rceil, m)$ -树，如 $(2, 3)$ -树、 $(3, 6)$ -树或 $(7, 13)$ -树等。

B-树的外部节点（external node）更加名副其实——它们实际上未必意味着查找失败，而可能表示目标关键码存在于更低层次的某一外部存储系统中，顺着该节点的指示，即可深入至下一级存储系统并继续查找。正因为如此，不同于常规的搜索树，如图8.11所示，在计算B-树高度时，还需要计入其最底层的外部节点。

例如，图8.12(a)即为一棵由9个内部节点、15个外部节点以及14个关键码组成的4阶B-树，其高度 $h = 3$ ，其中每个节点包含1~3个关键码，拥有2~4个分支。

作为与二叉搜索树等价的“扁平化”版本，B-树的宽度（亦即最底层外部节点的数目）往往远大于其高度。因此在以图形描述B-树的逻辑结构时，我们往往需要简化其中分支的画法，并转而采用如图(b)所示的紧凑形式。

另外，既然外部节点均同处于最底层，且深度完全一致，故在将它们省略之后，通常还不致造成误解。因此，还可以将B-树的逻辑结构，进一步精简为如图(c)所示的最紧凑形式。

由这种最紧凑的表示形式，也可同时看出，B-树叶节点（即最深的内部节点）的深度也必然完全一致，比如 $[7]$ 、 $[19, 22]$ 、 $[28]$ 、 $[37, 40, 41]$ 、 $[46]$ 和 $[52]$ 。

^④ 由R. Bayer和E. McCreight于1970年合作发明^[43]

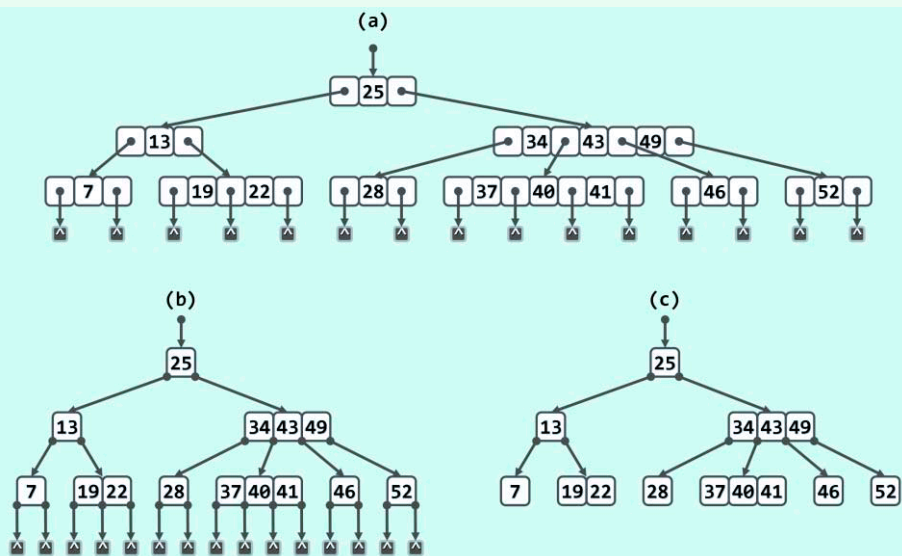


图8.12 (a) 4阶B-树；(b) B-树的紧凑表示；(c) B-树的最紧凑表示

8.2.2 ADT接口及其实现

按照以上定义，可以模板类的形式描述并实现B-树节点以及B-树结构本身如下。

■ 节点

B-树节点BTreeNode类，可实现如代码8.6所示。

```
1 #include "../vector/vector.h"
2 #define BTreeNodePosi(T) BTreeNode<T>* //B-树节点位置
3
4 template <typename T> struct BTreeNode { //B-树节点模板类
5     // 成员（为简化描述起见统一开放，读者可根据需要进一步封装）
6     BTreeNodePosi(T) parent; //父节点
7     Vector<T> key; //关键码向量
8     Vector<BTreeNodePosi(T)> child; //孩子向量（其长度总比key多一）
9     // 构造函数（注意：BTreeNode只能作为根节点创建，而且初始时有0个关键码和1个空孩子指针）
10    BTreeNode() { parent = NULL; child.insert(0, NULL); }
11    BTreeNode(T e, BTreeNodePosi(T) lc = NULL, BTreeNodePosi(T) rc = NULL) {
12        parent = NULL; //作为根节点，而且初始时
13        key.insert(0, e); //只有一个关键码，以及
14        child.insert(0, lc); child.insert(1, rc); //两个孩子
15        if (lc) lc->parent = this; if (rc) rc->parent = this;
16    }
17 };
```

代码8.6 B-树节点

这里，同一节点的所有孩子组织为一个向量，各相邻孩子之间的关键码也组织为一个向量。当然，按照B-树的定义，孩子向量的实际长度总是比关键码向量多一。

■ B-树

B-树模板类，可实现如代码8.7所示。

```

1 #include "BNode.h" //引入B-树节点类
2
3 template <typename T> class BTree { //B-树模板类
4 protected:
5     int _size; //存放的关键码总数
6     int _order; //B-树的阶次，至少为3——创建时指定，一般不能修改
7     BNodePosi(T) _root; //根节点
8     BNodePosi(T) _hot; //BTree::search()最后访问的非空（除非树空）的节点位置
9     void solveOverflow(BNodePosi(T)); //因插入而上溢之后的分裂处理
10    void solveUnderflow(BNodePosi(T)); //因删除而下溢之后的合并处理
11 public:
12    BTree(int order = 3) : _order(order), _size(0) //构造函数：默认为最低的3阶
13    { _root = new BNode<T>(); }
14    ~BTree() { if (_root) release(_root); } //析构函数：释放所有节点
15    int const order() { return _order; } //阶次
16    int const size() { return _size; } //规模
17    BNodePosi(T) & root() { return _root; } //树根
18    bool empty() const { return !_root; } //判空
19    BNodePosi(T) search(const T& e); //查找
20    bool insert(const T& e); //插入
21    bool remove(const T& e); //删除
22 }; //BTree

```

代码8.7 B-树

后面将会看到，B-树的关键码插入操作和删除操作，可能会引发节点的上溢和下溢。因此，这里设有内部接口solveOverflow()和solveUnderflow()，分别用于修正此类问题。在稍后的8.2.6节和8.2.8节中，将分别讲解其具体原理及实现。

8.2.3 关键码查找

■ 算法

如前述，B-树结构非常适宜于在相对更小的内存中，实现对大规模数据的高效操作。

一般地如图8.13所示，可以将大数据集组织为B-树并存放于外存。对于活跃的B-树，其根节点会常驻于内存；此外，任何时刻通常只有另一节点（称作当前节点）留驻于内存。

B-树的查找过程，与二叉搜索树的查找过程基本类似。

首先以根节点作为当前节点，然后再逐层深入。若在当前节点（所包含的一组关键码）中能够找到目标关键码，则成功返回。否则（在当前节点中查找“失败”），则必可在当前节点中确定某一个引用（“失败”位置），并通过它转至逻辑上处于下一层的另一节点。若该节点不是外部节点，则将其载入内存，并更新为当前节点，然后继续重复上述过程。

整个过程如图8.13所示，从根节点开始，通过关键码的比较不断深入至下一层，直到某一关键码命中（查找成功），或者到达某一外部节点（查找失败）。

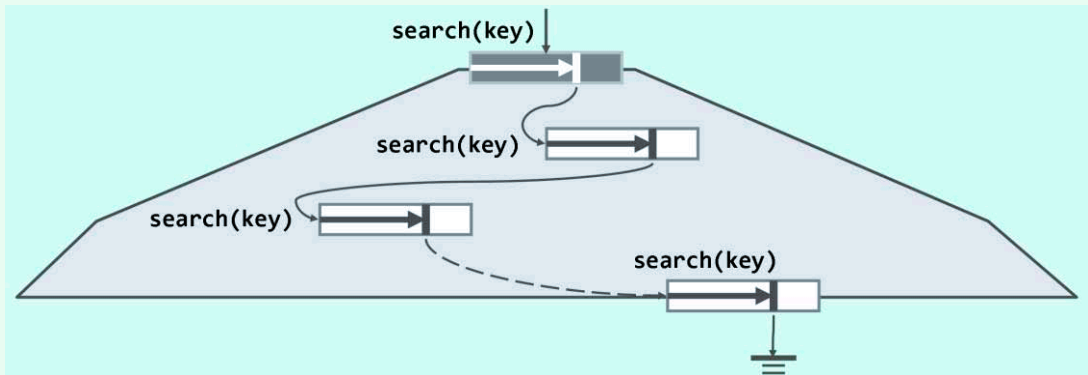


图8.13 B-树的查找过程

与二叉搜索树的不同之处在于，因此时各节点内通常都包含多个关键码，故有可能需要经过（在内存中的）多次比较，才能确定应该转向下一层的哪个节点并继续查找。

仍以如图8.12所示的4阶B-树为例，查找关键码41的过程大致如下：在根节点处经过一次关键码比较（25）之后，即可确定应转入第2个分支；再经过两次比较（34, 43）之后，确定转入第2个分支；最后经过三次比较（37, 40, 41）之后，才成功地找到目标关键码。查找关键码42的过程与之类似，只是在最底层的内部节点内，需要经过三次关键码比较（37, 40, 41）之后，才确定应转入关键码41右侧的外部节点，从而最终确定查找失败。

可见，只有在切换和更新当前节点时才会发生I/O操作，而在同一节点内部的查找则完全在内存中进行。因内存的访问速度远远高于外存，再考虑到各节点所含关键码数量通常在128~512之间，故可直接使用顺序查找策略，而不必采用二分查找之类的复杂策略。

■ 实现

如代码8.8所示，为简化代码，节点内部的查找直接借用了有序向量的search()接口。

```
1 template <typename T> BTreeNodePosi(T) BTree<T>::search(const T& e) { //在B-树中查找关键码e
2     BTreeNodePosi(T) v = _root; _hot = NULL; //从根节点出发
3     while (v) { //逐层查找
4         Rank r = v->key.search(e); //在当前节点中，找出不大于e的最大关键码
5         if ((0 <= r) && (e == v->key[r])) return v; //成功：在当前节点中命中目标关键码
6         _hot = v; v = v->child[r + 1]; //否则，转入对应子树（_hot指向其父）——需做I/O，最费时间
7     } //这里在向量内是二分查找，但对通常的_order可直接顺序查找
8     return NULL; //失败：最终抵达外部节点
9 }
```

代码8.8 B-树关键码的查找

与二叉搜索树的实现类似，这里也约定查找结果由返回的节点位置指代：成功时返回目标关键码所在的节点，上层调用过程可在该节点内进一步查找以确定准确的命中位置；失败时返回对应外部节点，其父节点则由变量_hot指代。

8.2.4 性能分析

由上可见，B-树查找操作所需的时间不外乎消耗于两个方面：将某一节点载入内存，以及在内存中对当前节点进行查找。鉴于内存、外存在访问速度上的巨大差异，相对于前一类时间消耗，后一类时间消耗可以忽略不计。也就是说，B-树查找操作的效率主要取决于查找过程中的外存访问次数。那么，至多需要访问多少次外存呢？

由前节分析可见，与二叉搜索树类似，B-树的每一次查找过程中，在每一高度上至多访问一个节点。这就意味着，对于高度为 h 的B-树，外存访问不超过 $O(h - 1)$ 次。

B-树节点的分支数并不固定，故其高度 h 并不完全取决于树中关键码的总数 n 。对于包含 N 个关键码的 m 阶B-树，高度 h 具体可在多大范围内变化？就渐进意义而言， h 与 m 及 N 的关系如何？

■ 树高

可以证明，若存有 N 个关键码的 m 阶B-树高度为 h ，则必有：

$$\log_m(N + 1) \leq h \leq \log_{\lceil m/2 \rceil} \lfloor (N + 1) / 2 \rfloor + 1 \dots\dots\dots \text{(式8-1)}$$

首先证明 $h \leq \log_{\lceil m/2 \rceil} \lfloor (N + 1) / 2 \rfloor + 1$ 。关键码总数固定时，为使B-树更高，各内部节点都应包含尽可能少的关键码。于是按照B-树的定义，各高度层次上节点数目至少是：

$$\begin{aligned} n_0 &= 1 \\ n_1 &= 2 \\ n_2 &= 2 \times \lceil m / 2 \rceil \\ n_3 &= 2 \times \lceil m / 2 \rceil^2 \\ &\dots \\ n_{h-1} &= 2 \times \lceil m / 2 \rceil^{h-2} \\ n_h &= 2 \times \lceil m / 2 \rceil^{h-1} \end{aligned}$$

现考查外部节点。这些节点对应于失败的查找，故其数量 n_h 应等于失败查找可能情形的总数，即应比成功查找可能情形的总数恰好多1，而后者等于关键码的总数 N 。于是有

$$N + 1 = n_h \geq 2 \times (\lceil m / 2 \rceil)^{h-1}, \quad h \geq 1$$

即 $h \leq 1 + \log_{\lceil m/2 \rceil} \lfloor (N + 1) / 2 \rfloor = O(\log_m N)$

再来证明 $h \geq \log_m(N + 1)$ 。同理，关键码总数固定时，为使B-树更矮，每个内部节点都应该包含尽可能多的关键码。按照B-树的定义，各高度层次上的节点数目至多是：

$$\begin{aligned} n_0 &= 1 \\ n_1 &= m \\ n_2 &= m^2 \\ &\dots \\ n_{h-1} &= m^{h-1} \\ n_h &= m^h \end{aligned}$$

与上同理，有

$$N + 1 = n_h \leq m^h$$

即 $h \geq \log_m(N + 1) = \Omega(\log_m N)$

总之，式8-1必然成立。也就是说，存有 N 个关键码的 m 阶B-树的高度 $h = \Theta(\log_m N)$ 。

■ 复杂度

因此，每次查找过程共需访问 $O(\log_m N)$ 个节点，相应地需要做 $O(\log_m N)$ 次外存读取操作。由此可知，对存有 N 个关键码的 m 阶B-树的每次查找操作，耗时不超过 $O(\log_m N)$ 。

需再次强调的是，尽管没有渐进意义上的改进，但相对而言极其耗时的I/O操作的次数，却已大致缩减为原先的 $1/\log_2 m$ 。鉴于 m 通常取值在256至1024之间，较之此前大致降低一个数量级，故使用B-树后，实际的访问效率将有十分可观的提高。

8.2.5 关键码插入

B-树的关键码插入算法，可实现如代码8.9所示。

```
1 template <typename T> bool BTree<T>::insert(const T& e) { //将关键码e插入B树中
2     BTreeNodePosi(T) v = search(e); if (v) return false; //确认目标节点不存在
3     Rank r = _hot->key.search(e); //在节点_hot的有序关键码向量中查找合适的插入位置
4     _hot->key.insert(r + 1, e); //将新关键码插至对应的位置
5     _hot->child.insert(r + 2, NULL); //创建一个空子树指针
6     _size++; //更新全树规模
7     solveOverflow(_hot); //如有必要，需做分裂
8     return true; //插入成功
9 }
```

代码8.9 B-树关键码的插入

为在B-树中插入一个新的关键码 e ，首先调用 $\text{search}(e)$ 在树中查找该关键码。若查找成功，则按照“禁止重复关键码”的约定不予插入，操作即告完成并返回 false 。

否则，按照代码8.8的出口约定，查找过程必然终止于某一外部节点 v ，且其父节点由变量 $_hot$ 指示。当然，此时的 $_hot$ 必然指向某一叶节点（可能同时也是根节点）。接下来，在该节点中再次查找目标关键码 e 。尽管这次查找注定失败，却可以确定 e 在其中的正确插入位置 r 。最后，只需将 e 插至这一位置。

至此， $_hot$ 所指的节点中增加了一个关键码。若该节点内关键码的总数依然合法（即不超过 $m - 1$ 个），则插入操作随即完成。否则，称该节点发生了一次上溢（overflow），此时需要通过适当的处理，使该节点以及整树重新满足B-树的条件。由代码8.9可见，这项任务将借助调整算法 $\text{solveOverflow}(_hot)$ 来完成。

8.2.6 上溢与分裂

■ 算法

一般地，刚发生上溢的节点，应恰好含有 m 个关键码。若取 $s = \lfloor m/2 \rfloor$ ，则它们依次为：

$$\{ k_0, \dots, k_{s-1}; k_s; k_{s+1}, \dots, k_{m-1} \}$$

可见，以 k_s 为界，可将该节点分前、后两个子节点，且二者大致等长。于是，可令关键码 k_s 上升一层，归入其父节点（若存在）中的适当位置，并分别以这两个子节点作为其左、右孩子。这一过程，称作节点的分裂（split）。

不难验证，如此分裂所得的两个孩子节点，均符合 m 阶B-树关于节点分支数的条件。

可能的情况

以如图8.14(a1)所示的6阶B-树局部为例，其中节点{ 17, 20, 31, 37, 41, 56 }，因所含关键码增至6个而发生上溢。为完成修复，可以关键码37为界，将该节点分裂为{ 17, 20, 31 }和{ 41, 56 }；关键码37则上升一层，并以分裂出来的两个子节点作为左、右孩子。

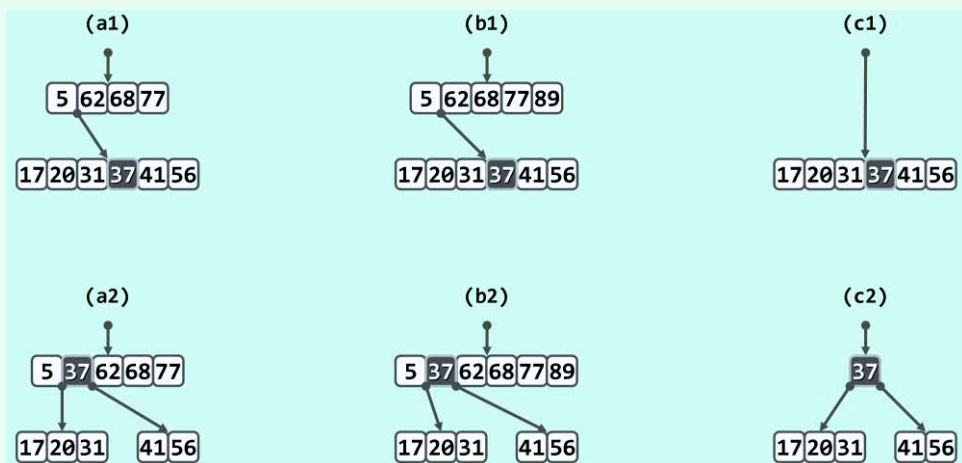


图8.14 通过分裂修复上溢节点

被提升的关键码，可能有三种进一步的处置方式。首先如图(a1)所示，设原上溢节点的父节点存在，且足以接纳一个关键码。此种情况下，只需将被提升的关键码(37)按次序插入父节点中，修复即告完成，修复后的局部如图(a2)所示。

其次如图(b1)所示，尽管上溢节点的父节点存在，但业已处于饱和状态。此时如图(b2)，在强行将被提升的关键码插入父节点之后，尽管上溢节点也可得到修复，却会导致其父节点继而发生上溢——这种现象称作上溢的向上传递。好在每经过一次这样的修复，上溢节点的高度都必然上升一层。这意味着上溢的传递不至于没有尽头，最远不至超过树根。

最后如图(c1)所示，若上溢果真传递至根节点，则可令被提升的关键码(37)自成一个节点，并作为新的树根。于是如图(c2)所示，至此上溢修复完毕，全树增高一层。可见，整个过程中所做分裂操作的次数，必不超过全树的高度——根据8.2.4节结论，即 $O(\log_m N)$ 。

实现

以上针对上溢的处理算法，可实现如代码8.10所示。

```
1 template <typename T> //关键码插入后若节点上溢，则做节点分裂处理
2 void BTree<T>::solveOverflow(BTNodePosi(T) v) {
3     if (_order >= v->child.size()) return; //递归基：当前节点并未上溢
4     Rank s = _order / 2; //轴点（此时应有_order = key.size() = child.size() - 1）
5     BTNodePosi(T) u = new BTNode<T>(); //注意：新节点已有一个空孩子
6     for (Rank j = 0; j < _order - s - 1; j++) { //v右侧的_order-s-1个孩子及关键码分裂为右侧节点u
7         u->child.insert(j, v->child.remove(s + 1)); //逐个移动效率低
8         u->key.insert(j, v->key.remove(s + 1)); //此策略可改进
9     }
10    u->child[_order - s - 1] = v->child.remove(s + 1); //移动v最靠右的孩子
```

```

11  if (u->child[0]) //若u的孩子非空, 则
12      for (Rank j = 0; j < _order - s; j++) //将它们的父节点统一
13          u->child[j]->parent = u; //指向u
14  BTreeNodePosi(T) p = v->parent; //v当前的父节点p
15  if (!p) { _root = p = new BTreeNode<T>(); p->child[0] = v; v->parent = p; } //若p为空则创建之
16  Rank r = 1 + p->key.search(v->key[0]); //p中指向u的指针的秩
17  p->key.insert(r, v->key.remove(s)); //轴点关键码上升
18  p->child.insert(r + 1, u); u->parent = p; //新节点u与父节点p互联
19  solveOverflow(p); //上升一层, 如有必要则继续分裂——至多递归O(logn)层
20 }

```

代码8.10 B-树节点的上溢处理

请特别留意上溢持续传播至根的情况：原树根分裂之后，新创建的树根仅含单关键码。由此也可看出，就B-树节点分支数的下限要求而言，树根节点的确应该作为例外。

■ 实例

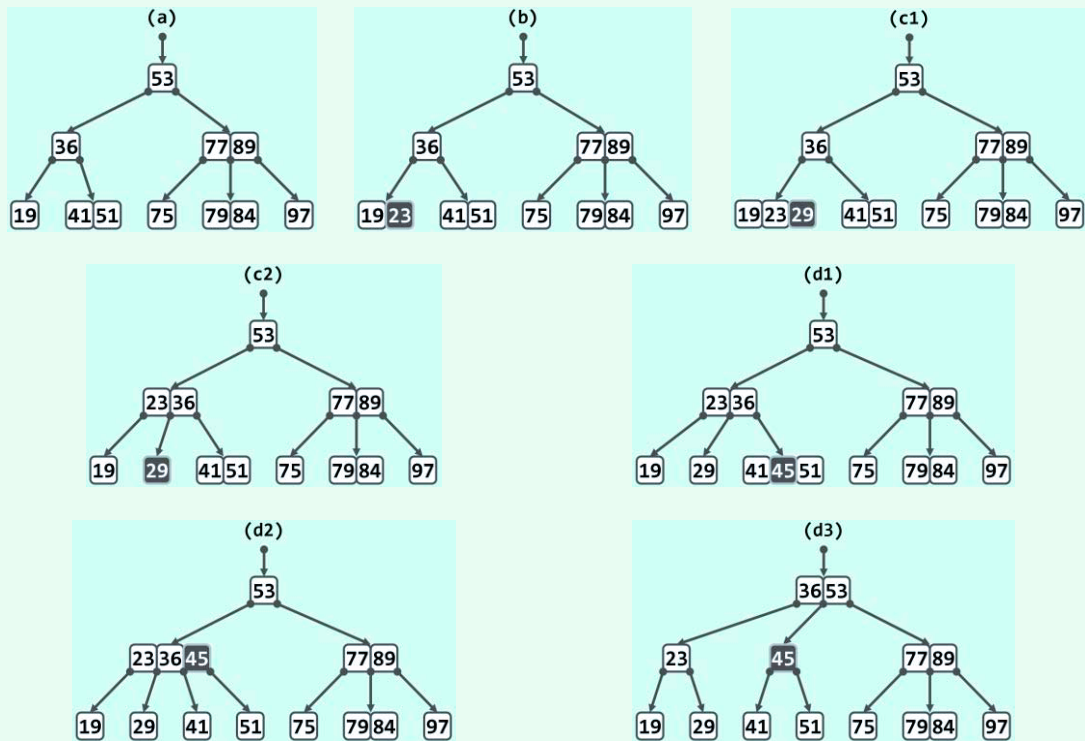


图8.15 3阶B-树插入操作实例 (I)

考查如图8.15(a)所示的3阶B-树。执行insert(23)后未发生任何上溢；如(b)所示不必做任何调整。接下来执行insert(29)后，如图(c1)所示发生上溢；经一次分裂即完全修复，结果如图(c2)所示。继续执行insert(45)后，如图(d1)所示发生上溢；经分裂做局部修复之后，如图(d2)所示上一层再次发生上溢；经再次分裂后，方得以实现全树的修复，结果如图(d3)所示。

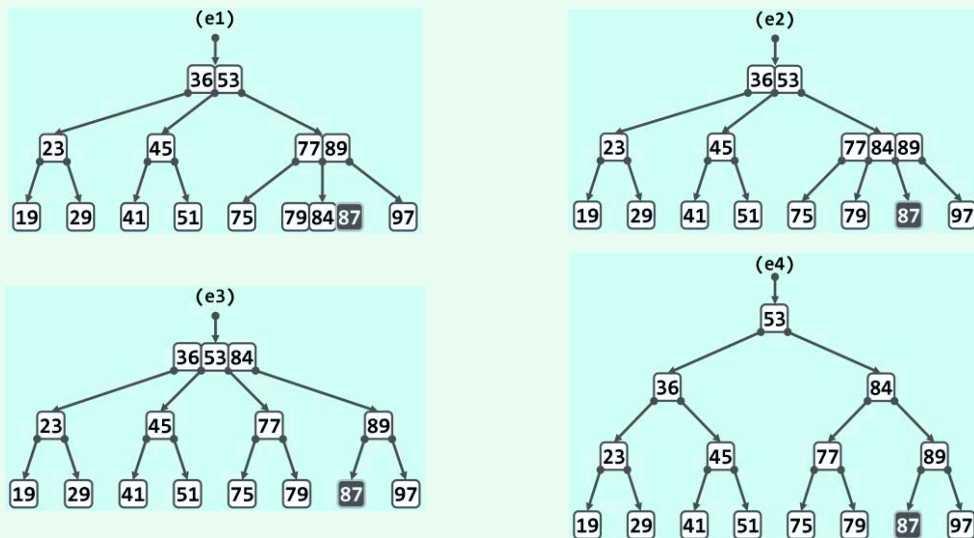


图8.16 3阶B-树插入操作实例 (II)

最后, 执行`insert(87)`后如图8.16(e1)所示亦发生上溢; 经局部分裂调整后, 在更高层将持续发生上溢, 故如图(e2)、(e3)和(e4)所示, 先后总共经三次分裂, 方得以实现全树的修复。此时因一直分裂至根节点, 故最终全树高度增加一层——这也是B-树长高的唯一可能。

■ 复杂度

若将B-树的阶次 m 视作为常数, 则关键码的移动和复制操作所需的时间都可以忽略。至于`solveOverflow()`算法, 其每一递归实例均只需常数时间, 递归层数不超过B-树高度。由此可知, 对于存有 N 个关键码的 m 阶B-树, 每次插入操作都可在 $O(\log_m N)$ 时间内完成。

实际上, 因插入操作而导致 $O(\log_m N)$ 次分裂的情况极为罕见, 单次插入操作平均引发的分裂次数, 远远低于这一估计 (习题[8-6]), 故时间通常主要消耗于对目标关键码的查找。

8.2.7 关键码删除

```

1 template <typename T> bool BTree<T>::remove(const T& e) {    //从BTree树中删除关键码e
2     BTreeNodePosi(T) v = search(e); if (!v) return false; //确认目标关键码存在
3     Rank r = v->key.search(e); //确定目标关键码在节点v中的秩 (由上, 肯定合法)
4     if (v->child[0]) {    //若v非叶子, 则e的后继必属于某叶节点
5         BTreeNodePosi(T) u = v->child[r+1]; //在右子树中一直向左, 即可
6         while (u->child[0]) u = u->child[0]; //找出e的后继
7         v->key[r] = u->key[0]; v = u; r = 0; //并与之交换位置
8     } //至此, v必然位于最底层, 且其中第r个关键码就是待删除者
9     v->key.remove(r); v->child.remove(r + 1); _size--; //删除e, 以及其下两个外部节点之一
10    solveUnderflow(v); //如有必要, 需做旋转或合并
11    return true;
12 }

```

代码8.11 B-树关键码的删除

B-树的关键码删除算法的实现如代码8.11所示。

为从B-树中删除关键码 e ，也首先需要调用 $\text{search}(e)$ 查找 e 所属的节点。倘若查找失败，则说明关键码 e 尚不存在，删除操作即告完成；否则按照代码8.8的出口约定，目标关键码所在的节点必由返回的位置 v 指示。此时，通过顺序查找，即可进一步确定 e 在节点 v 中的秩 r 。

不妨假定 v 是叶节点——否则， e 的直接前驱（后继）在其左（右）子树中必然存在，而且可在 $O(\text{height}(v))$ 时间内确定它们的位置，其中 $\text{height}(v)$ 为节点 v 的高度。此处不妨选用直接后继。于是， e 的直接后继关键码所属的节点 u 必为叶节点，且该关键码就是其中的最小者 $u[0]$ 。既然如此，只要令 e 与 $u[0]$ 互换位置，即可确保待删除的关键码 e 所属的节点 v 是叶节点。

于是，接下来可直接将 e （及其左侧的外部空节点）从 v 中删去。如此，节点 v 中所含的关键码以及（空）分支将分别减少一个。

此时，若该节点所含关键码的总数依然合法（即不少于 $\lceil m/2 \rceil - 1$ ），则删除操作随即完成。否则，称该节点发生了下溢（underflow），并需要通过适当的处置，使该节点以及整树重新满足B-树的条件。由代码8.11可见，这项任务将借助调整算法 $\text{solveUnderflow}(v)$ 来完成。

8.2.8 下溢与合并

由上，在 m 阶B-树中，刚发生下溢的节点 v 必恰好包含 $\lceil m/2 \rceil - 2$ 个关键码和 $\lceil m/2 \rceil - 1$ 个分支。以下将根据其左、右兄弟所含关键码的数目，分三种情况做相应的处置。

■ v 的左兄弟 L 存在，且至少包含 $\lceil m/2 \rceil$ 个关键码



图8.17 下溢节点向父亲“借”一个关键码，父亲再向左兄弟“借”一个关键码

如图8.17(a)所示，不妨设 L 和 V 分别是其父节点 P 中关键码 y 的左、右孩子， L 中最大关键码为 x （ $x \leq y$ ）。此时可如图(b)所示，将 y 从节点 P 转移至节点 V 中（作为最小关键码），再将 x 从 L 转移至 P 中（取代原关键码 y ）。至此，局部乃至整树都重新满足B-树条件，下溢修复完毕。

■ v 的右兄弟 R 存在，且至少包含 $\lceil m/2 \rceil$ 个关键码

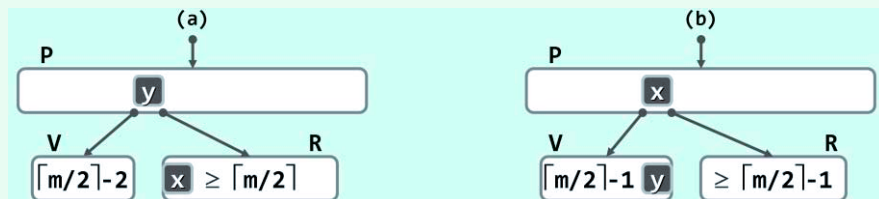


图8.18 下溢节点向父亲“借”一个关键码，父亲再向右兄弟“借”一个关键码

如图8.18所示，可参照前一情况对称地修复，不再赘述。

■ V的左、右兄弟L和R或者不存在，或者其包含的关键码均不足 $\lceil m/2 \rceil$ 个

实际上，此时的L和R不可能同时不存在。如图8.19(a)所示，不失一般性地设左兄弟节点L存在。当然，此时节点L应恰好包含 $\lceil m/2 \rceil - 1$ 个关键码。

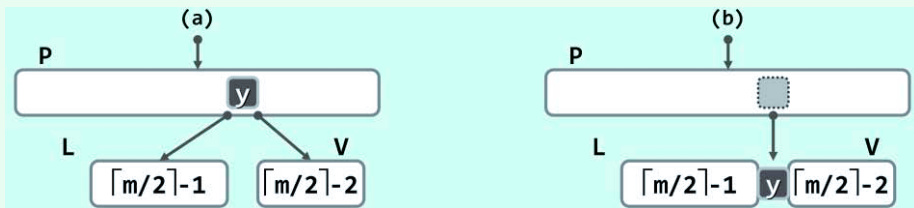


图8.19 下溢节点向父亲“借”一个关键码，然后与左兄弟“粘接”成一个节点

于是为修复节点V的下溢缺陷，可如图(b)所示，从父节点P中抽出介于L和V之间的关键码y，并通过该关键码将节点L和V“粘接”成一个节点——这一过程称作节点的合并(merge)。注意，在经过如此合并而得新节点中，关键码总数应为：

$$(\lceil m/2 \rceil - 1) + 1 + (\lceil m/2 \rceil - 2) = 2 \times \lceil m/2 \rceil - 2 \leq m - 1$$

故原节点V的下溢缺陷得以修复，而且同时也不致于反过来引发上溢。

接下来，还须检查父节点P——关键码y的删除可能致使该节点出现下溢。好在，即便如此，也尽可套用上述三种方法继续修复节点P。当然，修复之后仍可能导致祖父节点以及更高层节点的下溢——这种现象称作下溢的传递。特别地，当下溢传递至根节点且其中不再含有任何关键码时，即可将其删除并代之以其唯一的子节点，全树高度也随之下降一层。

与上溢传递类似地，每经过一次下溢修复，新下溢节点的高度都必然上升一层。再次由8.2.4节的式8-1可知，整个下溢修复的过程中至多需做 $O(\log_m N)$ 次节点合并操作。

■ 实现

对下溢节点的整个处理过程，如代码8.12所示。

```
1 template <typename T> //关键码删除后若节点下溢，则做节点旋转或合并处理
2 void BTree<T>::solveUnderflow(BTreeNodePosi(T) v) {
3     if (((_order + 1) / 2 <= v->child.size()) return; //递归基：当前节点并未下溢
4     BTreeNodePosi(T) p = v->parent;
5     if (!p) { //递归基：已到根节点，没有孩子的下限
6         if (!v->key.size() && v->child[0]) {
7             //但倘若作为树根的v已不含关键码，却有（唯一的）非空孩子，则
8             _root = v->child[0]; _root->parent = NULL; //这个节点可被跳过
9             v->child[0] = NULL; release(v); //并因不再有用而被销毁
10        } //整树高度降低一层
11        return;
12    }
13    Rank r = 0; while (p->child[r] != v) r++;
14    //确定v是p的第r个孩子——此时v可能不含关键码，故不能通过关键码查找
15    //另外，在实现了孩子指针的判等器之后，也可直接调用Vector::find()定位
16    //情况1：向左兄弟借关键码
```